Needs-Based Homeostatic Agent Model

1. Overview

This model implements a **Needs-Based Homeostatic Agent** grounded in Maslow's prepotency hierarchy. Agents on a discrete torus grid survive by satisfying two biological needs — energy (food) and hydration (water). Decision logic follows a strict priority function: more urgent needs always preempt less urgent ones.

Why this model ?

Property	Why it matters
Continuous state decay	Directly exercises the missing primitive identified in Mesa Discussion #2529: Continuous States
Priority-based decisions	Requires explicit `if/elif` chains — quantifiable cyclomatic complexity
Reproduction with cooldown	Tests multi-tick state management without a duration primitive
Spatial foraging	Exercises `cell.get_neighborhood()` and greedy movement

---

2. Variable Definitions

Agent State

Symbol	Name	Python attribute	Range	Description
$E(t)$	Energy	`self.energy`	$[0, E_{\max}]$	Caloric reserve
$H(t)$	Hydration	`self.hydration`	$[0, H_{\max}]$	Water reserve
$C(t)$	Reproduction Cooldown	`self._repro_cooldown`	$[0, \tau]$	Steps until reproduction allowed
$\mathbf{p}(t)$	Position	`self.cell`	Grid $[0, W) \times [0, L)$	Current cell (`Cell` type)

Biological Parameters

Symbol	Parameter	Default	Description
λ_E	Energy decay rate	1.0	Energy lost per step
λ_H	Hydration decay rate	1.5	Hydration lost per step (faster than energy)
v	Vision radius	3	Moore neighbourhood search radius
ΔE_{cost}	Reproduction energy cost	15	Energy deducted on reproduction
ΔH_{cost}	Reproduction hydration cost	10	Hydration deducted on reproduction

Symbol	Parameter	Default	Description
τ	Reproduction cooldown	5	Steps before next reproduction allowed

Decision Thresholds

Symbol	Name	Default	Meaning
θ_H	Hydration critical threshold	30	Below this → forage water
θ_E	Energy critical threshold	30	Below this → forage food
ϕ_H	Hydration thriving threshold	70	Above this (with ϕ_E) → reproduce
ϕ_E	Energy thriving threshold	70	Above this (with ϕ_H) → reproduce

Environment Parameters

Parameter	Default	Description
Grid width W	25	Cells
Grid height L	25	Cells
Initial population N	50	Agents
Food patch density	0.15	Fraction of cells seeded with food
Water patch density	0.10	Fraction of cells seeded with water
Food regrowth time T_F	8	Steps until eaten food regrows
Water regrowth time T_W	12	Steps until consumed water replenishes
Food energy value	40	Energy gained per food patch
Water hydration value	50	Hydration gained per water patch

---

3. Formal Equations

3.1 Biological Homeostasis (State Decay)

$$E(t+1) = \max(0,\ E(t) - \lambda_E) \tag{1}$$

$$H(t+1) = \max(0,\ H(t) - \lambda_H) \tag{2}$$

$$C(t+1) = \max(0,\ C(t) - 1) \tag{3}$$

Mesa implementation. Three explicit assignments at the top of `NeedsAgent.step()`:

```
self.energy          = max(0.0, self.energy    - self.decay_energy)
self.hydration       = max(0.0, self.hydration - self.decay_hydration)
self._repro_cooldown = max(0.0, self._repro_cooldown - 1)
```

Pain point [P1]: Every agent subclass that needs a decaying variable must repeat this pattern, one line per variable. There is no primitive to express "this variable decays at rate λ per time unit." Mesa Discussion #2529 proposes solving this.

3.2 Mortality (Survival Threshold)

$$S(t) = \begin{cases} 1 & \text{if } E(t) > 0 \wedge H(t) > 0 \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

When $S(t) = 0$ the agent calls `self.remove()`.

Pain point [P2]: The check runs every tick regardless of energy level. There is no threshold-triggered callback; the condition is polled even when energy is at 99.

3.3 Spatial Pathfinding (Moore Neighbourhood)

Observable set with Moore neighbourhood, radius v:

$$P_{\text{obs}} = \{\mathbf{p}' : \max(|x' - x|,\ |y' - y|) \leq v\} \tag{5}$$

Nearest resource within P_{obs}:

$$\mathbf{p}_{\text{target}} = \underset{\mathbf{p}' \in P_{\text{obs}}}{\operatorname{argmin}} \sqrt{(x' - x)^2 + (y' - y)^2} \tag{6}$$

Move one step toward $\mathbf{p}_{\text{target}}$ (greedy):

$$\mathbf{p}(t+1) = \underset{\mathbf{q} \in \mathcal{N}_1(\mathbf{p})}{\operatorname{argmin}} \|\mathbf{q} - \mathbf{p}_{\text{target}}\|_2 \tag{7}$$

where $\mathcal{N}_1(\mathbf{p})$ is the set of radius-1 Moore neighbours. If no resource exists within P_{obs}, the agent wanders randomly.

3.4 Reproduction Cost (Equations 8–13)

$$E_{\text{parent}}(t+1) = E_{\text{parent}}(t)/2 - \Delta E_{\text{cost}} \tag{8}$$

$$H_{\text{parent}}(t+1) = H_{\text{parent}}(t)/2 - \Delta H_{\text{cost}} \tag{9}$$

$$C_{\text{parent}}(t+1) = \tau \tag{10}$$

Offspring spawned on parent's cell with:

$$E_{\text{child}} = E_{\text{parent}}(t)/2 \tag{11}$$

$$H_{\text{child}} = H_{\text{parent}}(t)/2 \tag{12}$$

$$C_{\text{child}} = \tau \tag{13}$$

The offspring inherits λ_E, λ_H, v, and all thresholds.

3.5 The Decision Matrix (Prepotency)

$$A(t) = \begin{cases} \text{ForageWater} & \text{if } H(t) < \theta_H \quad \text{[highest urgency]} \\ \text{ForageFood} & \text{if } E(t) < \theta_E \\ \text{Reproduce} & \text{if } E(t) \geq \phi_E \wedge H(t) \geq \phi_H \wedge C(t) = 0 \\ \text{Wander} & \text{otherwise} \end{cases} \tag{14}$$

Pain point [P3]: Priority is encoded positionally in the `if/elif` chain. Adding a new behaviour requires finding the correct insertion point in a growing chain. The ordering is implicit and not self-documenting.

---

4. Environment Dynamics

`FoodPatch` and `WaterPatch` are `FixedAgent` subclasses. Both `consume()` methods guard with `if self.has_food/has_water` before scheduling regrowth, making them **idempotent**: safe to call more than once within a step without double-scheduling a regrowth event.

`schedule_event` is the correct Mesa primitive for timed regrowth. This is already idiomatic Mesa and the behavioral framework adds nothing here. This is an important Phase 1 finding: not all pain points need framework solutions.

---

5. Documented Pain Points

#	Pain Point	Code location	Expected solution
P1	Manual decay — one `max(0, x - )` line per variable	`NeedsAgent.step()` lines 1–3	`BehavioralState(decay_rate= )`
P2	Survival check polled every tick regardless of energy level	`NeedsAgent.step()` line 4	`BehavioralState(thresholds={"d 0})`
P3	Priority encoded by `if/elif` insertion order — fragile	`NeedsAgent.step()` decision tree	`@rule(priority=RulePriority.UR`

#	Pain Point	Code location	Expected solution
P4	Cooldown is a raw float with manual decrement and clamp	`self._repro_cooldown`	`BehavioralState(decay_rate=-1, min_value=0)`
P5	Decay, survival, and decision logic entangled in one method	All of `NeedsAgent.step()`	Separate `@rule` methods + `DecisionSystem`

---

6. Birth/Death Accounting

Birth and death counts are computed in `NeedsBasedModel.step()`:

Births: `NeedsAgent._reproduce()` sets `self.gave_birth_this_step = True` on the parent. The model loop increments `_births_this_step` inline as each agent finishes its step.

Deaths: Computed via population delta after all agents have stepped:

$$\text{deaths} = (\text{pop\_start} + \text{births}) - \text{pop\_end} \tag{15}$$

---

7. References

- Maslow, A. H. (1943). A theory of human motivation. *Psychological Review*, 50(4), 370.
- Epstein, J. M. & Axtell, R. (1996). *Growing Artificial Societies*. MIT Press.
- Mesa Discussion #2529: Continuous States
- Mesa Discussion #2538: Behavioral Framework
- Mesa Discussion #3304: Actions — Event-driven Agent Behavior